

9. AI Tool Usage

This entire document was produced in an interactive session with **Claude Code** (Anthropic’s CLI coding agent, Sonnet 5), used as a drafting and design-review partner rather than a one-shot generator. This section documents how, honestly including where it fell short.

9.1 How it was used

- **Drafting from a rough sketch.** The starting point was a short, informal dump of entities, a candidate API list, and a few named patterns (CQRS+ES, Transactional Outbox, decoupled scheduler). Claude Code turned that into the initial Architecture Overview — component breakdown, Kafka topic table, sequence diagrams, entity model — as a first draft to react to, not a final answer.
- **Iterative design review, turn by turn.** Nearly every subsequent change came from a specific critique of the current draft (“should Debezium really be drawn inside the service box?”, “does the Dispatcher/Initiator split need its own diagram?”, “why no identity field on Order?”) rather than a fresh generation request. The tool proposed 2–3 concrete options with trade-offs at each real fork (e.g. scheduler design, notification topic shape, service topology) and implemented whichever was chosen.
- **Cross-document synthesis.** Sections 2 (Key Design Decisions), 3 (Trade-offs), 4 (Risks & Mitigations), 5 (Assumptions & Open Questions), 6 (NFRs), 7 (MVP/Post-MVP), and 8 (Delivery Plan) were each generated by synthesizing decisions already made during the Architecture Overview conversation into the shape the brief asks for — not independently re-derived, which is part of why they stay consistent with the diagrams rather than drifting from them.
- **Mechanical consistency maintenance.** Renaming components mid-conversation (e.g. an early “Due-Plan Enumerator” became “Execution Dispatcher”), replacing Postgres/WAL-specific language with vendor-neutral terms, renumbering sections after a diagram split, and a final grep-driven pass verifying every numbered cross-reference (5.1 #2, 7.3, etc.) across all files actually resolves to a real item.

9.2 What worked well

- **Fast resolution of genuine forks.** Presenting real architectural decisions (not cosmetic ones) as a short set of concrete options with a stated recommendation, then implementing the choice immediately, kept the session moving without long back-and-forth prose.
- **Catching structural gaps through review, not just generation.** Several real design gaps surfaced specifically *because* a human was reviewing closely and pushing back — e.g. that **Order** had no identity field despite 1.1 claiming events are “matched by correlation id,” that the Dispatcher’s crash-recovery mechanism needed to be distinguished from a poison-pill scenario, that **Entered** → **Active** has no designed trigger. The tool didn’t invent these on its own initiative; it responded well once they were raised.
- **Mermaid diagrams as a first-class, iterable artifact.** Diagrams were corrected with the same precision as code — “move this node outside the subgraph,” “split this into two diagrams at the async boundary” — and applied directly, avoiding a round-trip through an external diagramming tool.

9.3 What didn’t work well / limitations

- **Default assumptions needed active correction.** Early drafts used Postgres- and WAL-specific language despite the brief explicitly saying not to assume a tech stack, and initially placed Debezium as if it were a sub-component of the service rather than separately-deployed shared infrastructure. Neither was self-corrected — both required the user to notice and flag them.
- **Consistency decayed quietly across many incremental edits.** As sections got renumbered, diagrams got split, and terminology got renamed, stale cross-references and one reverted diagram relationship (a class-diagram edge that silently flipped from aggregation back to composition) accumulated without being caught in the moment. It took a deliberate, explicit “review everything” pass — not the normal edit-and-move-on flow — to find and fix them. In a longer-running session across many linked

documents, this kind of drift risk is real and worth budgeting review time for, not just trusting the tool to keep everything in sync as it goes.

- **No authority over real business facts.** The tool has no way to know what `Investment.quantity` actually means, whether `Trading/Orders` exposes a structured rejection reason code, or what BUX's actual availability targets are. It handled this appropriately — flagging these as stated assumptions and open questions (5, throughout) rather than guessing at confident-sounding answers — but that's a discipline the user had to keep checking for, not a guarantee the tool provides on its own.

9.4 Net assessment

Effective as an accelerant for structuring and cross-referencing a multi-section design document, and genuinely useful as a review partner once pushed on specific details — but it does not substitute for a domain expert's judgment or catch its own defaults and drift unprompted. The document reflects many rounds of human-directed correction, not a single generation accepted as-is.